

Fakultet tehničkih nauka
Univerzitet u Novom Sadu

Paralelne i distribuirane arhitekture i jezici

Uvod u Rust

Branislav Ristić

Zimski semestar, školska 2026./27.

Uvod

O predmetu

- Asistent: Branislav Ristić
  - NTP-321
  - branislav.ristic@uns.ac.rs
  - Konsultacije: dogovor putem elektronske pošte.
- Sajt predmeta:
  - https://www.acs.uns.ac.rs/sr/pidaij (obaveštenja, prezentacije)
  - https://ristic-ac.github.io/pdaj-materijali/ (plan rada, slajdovi i materijali za preuzimanje)
  - Pratiti redovno



Slika 1: Materijali na sajtu

Pregled tehnologija

- Operativni sistem:
  - MI-A2-3 - Ubuntu 20.04
  - NTP-312 - Debian 13
- Editor koda: VSCode
- Kompajler: rustc 1.75.0
- Upravljač projektom i zavisnostima: cargo 1.75.0
- Ekstenzije: rust analyzer (latest)

Specifikacije hardvera

- Minimalno:
  - CPU: 2 jezgra
  - RAM: 8 GB
  - Disk: SSD
- Preporučeno:
  - 6–12 jezgara (+ SMT)
  - 16 GB dual-channel
  - NVMe SSD
- Lakši život ako se ima:
  - 8–16 jezgra (+ SMT)
  - 32 GB+ (DDR5)

  - brzi NVMe SSD (PCIe v4/v5)

Rust

- Programski jezik opšte namene
  - Statički tipiziran, kompajlira se u mašinski kod.
- Fokusira se na:
  - Performanse:
    * Ne poseduje standardni garbage collector (ownership model).
    * Ne poseduje run-time environment.
  - Pouzdanost:
    * Detekcija grešaka prilikom kompajliranja.
    * Memorijska bezbednost bez garbage collector-a.
  - Produktivnost:
    * Ekosistem bogat alatima.

Primena Rust-a

- Operativni sistemi:
  - Redox
- Embedded sistemi:
  - IoT uređaji
- Web programiranje:
  - Wasm
- Blokčejn i kriptografija:
  - Solana pametni ugovori
- Video igre:
  - ECS sistemi
  - Game engine

Instalacija i Cargo

Instalacija Rust-a (Linux)

- Neophodni alati:
  - curl
- Komanda za instalaciju (rustup):

```
$ curl --proto '=https' --tlsv1.2 https://sh.rustup.rs -sSf | sh
```

- U slučaju uspešne instalacije biće prikazana poruka:
  - Rust is installed now. Great!
- Za proveru verzije koristi komandu:

```
$ rustc --version
```

- Za kompajliranje koristiti komandu:

```
$ rustc <naziv-izvorne-datoteke.rs>
```

Verzija Rusta

- Na kursu se koristi Rust 1.75.0.
- Instalacija i izbor verzije (rustup):

```
$ rustup toolchain install 1.75.0
$ rustup default 1.75.0
$ rustc --version
```

- Samo za tekući folder: `rustup override set 1.75.0`.
- Jednokratno: `cargo +1.75.0 build`.
- Instalirane verzije: `rustup show`.

Cargo

- Alat za:
  - Upravljanje projektom.
  - Razrešavanje zavisnosti.
- Ukoliko je Rust instaliran putem rustup-a, cargo bi trebao da je već instaliran, provera putem:

```
$ cargo --version
```

Kreiranje Cargo projekta (paketa)

- Komanda za kreiranje projekta:

```
$ cargo new <project_name>          # binarni projekat (--bin,
 ↪  podrazumevano)
$ cargo new --lib <project_name>    # projekat biblioteke
```

- Razlika između tipova projekata:
  - lib
    - Bibliotečki projekat.
    - Entrypoint tipično: lib.rs.
    - Ne poseduje main funkciju.
    - Rezultat kompajliranja .so ili .rlib datoteke.
  - bin
    - Izvršni projekat.
    - Entrypoint tipično: main.rs.
    - Poseduje main funkciju.
    - Rezultat kompajliranja binarne (izvršne) datoteke.

Kreiranje Cargo projekta (paketa)

- Nakon pokretanja komande:

```
$ cargo new hello_cargo
```

- Kreira se direktorijum sa sledećom strukturom:

```
hello_cargo/
├── .gitignore
├── Cargo.toml
└── src/
    └── main.rs
```

Hello World

```
// main.rs
fn main() {
    println!("Hello, world!");
}
```

- `fn main` je ulazna tačka programa.
- `println!` je makro (zbog `!`), a ne funkcija.
- `cargo run` ispisuje: `Hello, world!`

Cargo.toml

- TOML - Tom's Obvious, Minimal Language.
- Konfiguracioni format:
  - Koristi ga i Cargo.

```
# FileName: Cargo.toml
[package]
name = "hello_cargo"    # Naziv projekta
version = "0.1.0"       # Verzija projekta
edition = "2021"        # Major release year

[dependencies]
tokio = { version = "1.32", features = ["full"] }
```

Verzije zavisnosti

- Verzija bez operatora znači ^: dozvoljene su kompatibilne novije verzije (semver).

```
tokio = "1.32"     # isto što i "^1.32": >= 1.32.0, < 2.0.0
serde = "=1.0.190" # tačno ta verzija
```

- `cargo add <crate>` dodaje zavisnost u `Cargo.toml`.

`cargo add` na Rustu 1.75

- Cargo 1.75 uzima najnoviju verziju crate-a, čak i ako traži noviji `rustc`.
  - Na primer, `cargo add rayon` upisuje `rayon = "1.12.0"`, a ta verzija traži Rust 1.80.
- `cargo add` uspe, a greška se javlja tek pri `cargo build`:

```
error: package `rayon v1.12.0` cannot be built because it requires rustc
 ↪  1.80 or newer, while the currently active rustc version is 1.75.0
```

- Isto važi i za zavisnosti tog crate-a: `rayon` koristi `rayon-core`, koji je takođe preuzet u verziji 1.13.0 (traži Rust 1.80).

Ručni izbor verzija

- Sniziti zahtev u `Cargo.toml`, pa svaki crate posebno spustiti na verziju koja radi na 1.75:

```
rayon = "1.10"
```

```
$ cargo update rayon --precise 1.10.0
$ cargo update rayon-core --precise 1.12.1
```

- `cargo update <crate>` ne menja zavisnosti tog crate-a (osim ako mora).
  - Komandu ponoviti za svaku stavku koju cargo prijavi.
- Novije verzije cargo-a mogu da biraju kompatibilne verzije: `cargo add` poštuje `rust-version` od 1.79.

Kompajliranje projekta

```
$ cargo build [--release]
```

- Rezultat se nalazi u `target/`:
  - `target/debug/<ime>` (podrazumevano),
  - `target/release/<ime>` (`--release`, optimizovano).

- Cargo.lock
  - Fajl u okviru kog se vodi evidencija o tačnim verzijama zavisnosti u projektu.
  - Ne menjati ručno.

Verzija Rusta i Cargo.lock

- `rust-version = "1.75"` u `[package]` navodi najstariji podržan kompajler.
  - Cargo prijavljuje grešku ako je kompajler stariji.
- `Cargo.lock` se obično čuva u repozitorijumu kod aplikacija i primera.
  - Svi dobijaju iste verzije zavisnosti.
  - Bez njega se uzimaju najnovije dozvoljene verzije, koje mogu da traže noviji `rustc`.
- `cargo build --locked` prijavljuje grešku umesto da izmeni `Cargo.lock`.

Pokretanje projekta

```
$ cargo run
```

- Pokreće aplikaciju:
  - Provera da li postoje izmene u kodu:
    - Postoje - cargo build, potom pokretanje.
    - Ne postoje - pokretanje programa.

Provera projekta

```
$ cargo check
```

- Proverava da li se kod kompajlira, tako da:
  - Ne kreira izvršni fajl.

Korisne cargo komande

Komanda	Namena
`cargo fmt`	formatira kod
`cargo clippy`	upozorenja i predlozi za bolji kod
`cargo test`	pokreće testove
`cargo update`	ažurira `Cargo.lock`
`cargo tree`	stablo zavisnosti
`cargo clean`	briše `target/`
`cargo help [command]`	pomoć za komandu

- `cargo fmt` i `cargo clippy` mogu da zahtevaju dodatne komponente: `rustup component add rustfmt clippy`.

Organizacija koda

Crate

- Najmanja jedinica programskog prevođenja u Rust-u.
- Vrste crate-ova:
  - Binarni,
  - Bibliotečki.
- Moguće je preuzeti već gotove crate-ove sa crates.io.
- Primer:
  - main.rs u okviru src/ (binarni),
  - lib.rs (bibliotečki).

Paket

- Paket predstavlja skup jednog ili više crate-ova:
  - Potencijalno više binarnih crate-ova [0,N],
  - Najviše jedan bibliotečki crate [0,1].
- Paket poseduje Cargo.toml koji opisuje kako se crate-ovi i njihove zavisnosti build-uju.
- Paket se sam po sebi ne kompajlira, već služi kako bi se upravljalo sa više crate-ova odjednom.
- Primer:
  - U okviru paketa se nalaze crate-ovi:
    - src/main.rs
    - src/lib.rs (max 1)
    - Potencijalno dodatni bin crate-ovi u okviru src/bin/

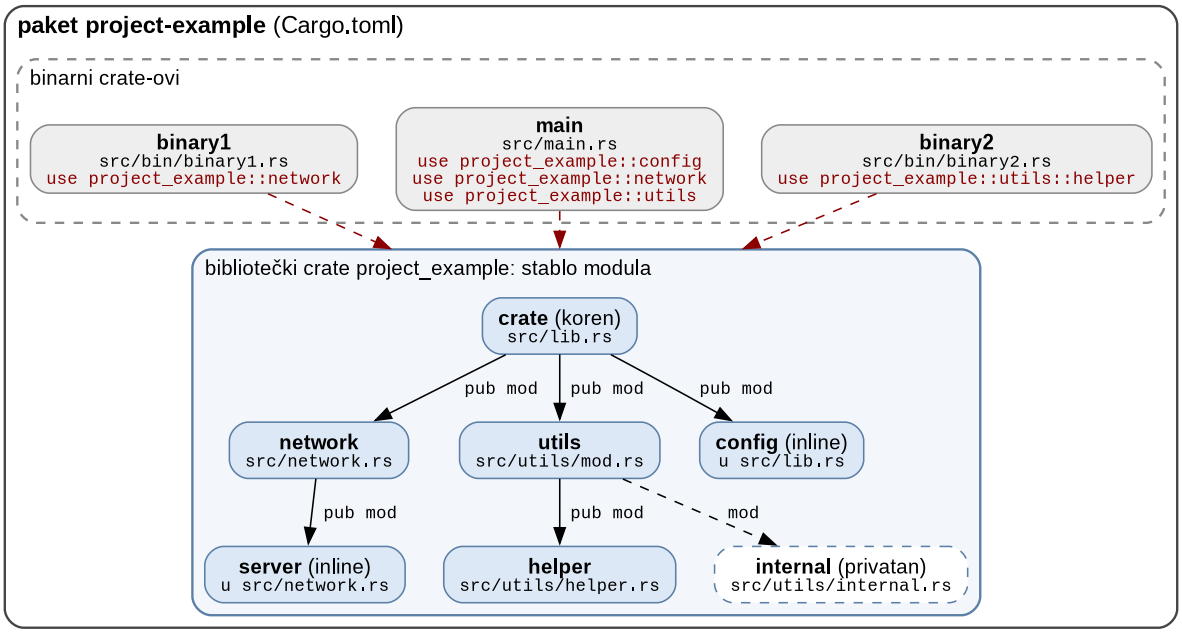
Modul

- Moduli pružaju mogućnost hijerarhijske organizacije koda u okviru crate-a.
- Svaki crate ima korenski modul (main.rs ili lib.rs).
- Moguće je:
  - Ugnježdavati module.
  - Deliti module u različite fajlove.

Modul: primer

```
project-example/
├── Cargo.toml
└── src/
    ├── main.rs
    ├── lib.rs
    ├── bin/
    │   ├── binary1.rs
    │   ├── binary2.rs
    │   ├── visibility.rs
    │   └── nocompile_private_module.rs
    ├── network.rs
    └── utils/
        ├── mod.rs
        ├── helper.rs
        └── internal.rs
```

Modul: crate-ovi i stablo modula



Slika 2: Crate-ovi i moduli paketa project-example

Primer: lib.rs

primeri/intro-to-rust/project-example/src/lib.rs

```rust
// A module can be defined in three ways: as src/<name>.rs, as
 ↪  src/<name>/mod.rs, or inline.
// Having both src/<name>.rs and src/<name>/mod.rs is an error.
pub mod network; // File module: here it is src/network.rs (the 2018
 ↪  style; its sub-module is inline)
pub mod utils; // File module: here it is src/utils/mod.rs (the older
 ↪  style)

// Inline module: the body is written in place (curly brackets instead of
 ↪  a semicolon), no extra file
pub mod config {
    pub fn print_config() {
        println!("Config module defined inline in lib.rs.");
    }
}

// Removing `pub` would make a module private to the crate and
 ↪  inaccessible from outside, which in
// turn means the main binary crate would not be able to use it, i.e., it
 ↪  would cause compilation errors in `src/main.rs`.
```

Primer: network.rs

primeri/intro-to-rust/project-example/src/network.rs

```rust
// This file defines the "network" module and its sub-module "server".
// It is a file module (src/network.rs). Its sub-module "server" is
 ↪  defined inline, so no extra file is needed.
```

```rust

// Public function in the "network" module
pub fn connect() {
    println!("Connecting to the network...");
}

// Inline sub-module: the body is written in place (curly brackets instead
 ↪  of a semicolon)
pub mod server {
    use crate::utils::helper; // Absolute path from the crate root:
     ↪  "utils" is in the same crate (library), so we start with crate::

    // A function in the "server" module
    pub fn start_server() {
        println!("Server started from the network::server module!");
        // Call a function from another module of the same crate
        helper::print_help();
    }
}
```

Primer: utils/mod.rs

primeri/intro-to-rust/project-example/src/utils/mod.rs

```rust
// This file defines the "utils" module and its sub-modules "helper" and
 ↪  "internal".

pub mod helper; // Public sub-module: the compiler looks for it in
 ↪  src/utils/helper.rs
mod internal; // Private sub-module (no pub): visible only in utils and
 ↪  its descendants

// Public function of the "utils" module; it may use the private module
 ↪  because it is in the same module
pub fn print_hello() {
    println!("{}", internal::decorate("Hello from the utils module!"));
}
```

Primer: utils/helper.rs

primeri/intro-to-rust/project-example/src/utils/helper.rs

```rust
use crate::network; // Absolute path from the crate root: "network" is a
 ↪  sibling of "utils" in the same crate (library).

// A helper function within the "helper" module
pub fn print_help() {
    // "internal" is private, but helper may use it: a private module is
     ↪  visible in its parent (utils)
    // and in all of the parent's descendants, including this sibling
     ↪  module
    println!("{}", super::internal::decorate("Helper function from the
     ↪  utils::helper module!"));
    // Call a function from another module of the same crate
    network::connect();
}
```

Primer: utils/internal.rs

primeri/intro-to-rust/project-example/src/utils/internal.rs

```rust
// Private module: utils/mod.rs declares it with `mod internal;` (no pub).
// The function is pub, but it is still not reachable outside the library,
 ↪  because the module is not pub.
pub fn decorate(text: &str) -> String {
    "** ".to_string() + text + " **"
}
```

Primer: main.rs

primeri/intro-to-rust/project-example/src/main.rs

```rust
// Use the library crate defined in lib.rs
use project_example::config;
use project_example::network;
use project_example::utils;

fn main() {
    println!("This is the main binary crate.");
    // Call functions from the "utils" module
    utils::print_hello();
    utils::helper::print_help();

    // Call a function from the inline "config" module
    config::print_config();

    // Call functions from the "network" module
    network::connect();
    network::server::start_server();
}
```

Izlaz

```
This is the main binary crate.
** Hello from the utils module! **
** Helper function from the utils::helper module! **
Connecting to the network...
Config module defined inline in lib.rs.
Connecting to the network...
Server started from the network::server module!
** Helper function from the utils::helper module! **
Connecting to the network...
```

Modul: tri načina definisanja

Način	Deklaracija	Telo modula	Podmoduli
Fajl `<ime>.rs`	`mod network;`	`src/network.rs`	inline (`server`) ili `network/<ime>.rs`
Fajl `<ime>/mod.rs`	`mod utils;`	`src/utils/mod.rs`	fajlovi (`utils/helper.rs`)
Inline	`mod config { ... }`	u `lib.rs`	inline ili `mod x;`

- Deklaracija sa `;` (`mod network;`) učitava telo iz fajla.
- Za isti modul ne mogu da postoje oba fajla (`network.rs` i `network/mod.rs`): `E0761`.

- Putanja u kodu (`network::server::start_server`) i pravila privatnosti ne zavise od načina.

Modul: `pub mod` i `mod`

- Bez `pub` modul je privatan: vide ga samo njegov roditelj (modul u kom je deklarisan) i svi potomci roditelja.
  - `utils` i njegov podmodul `helper` smeju da koriste `internal`; koren crate-a, `network` i binarni crate-ovi ne smeju.
- Binarni crate vidi biblioteku samo kroz javni API: svaki nivo puta mora biti `pub` (`pub mod`, `pub fn`).
  - `pub mod` otvara samo put do modula; stavke u njemu se posebno označavaju sa `pub`.
- `pub(crate)`: vidljivo u celom crate-u, ali ne i van njega (ni u binarnim crate-ovima).

Modul: `pub mod` naspram `pub fn`

Putanja iz binarnog crate-a	Modul	Funkcija	Rezultat
`utils::print_hello()`	`pub mod utils`	`pub fn`	radi
`utils::internal::decorate()`	`mod internal`	`pub fn`	`E0603`: modul je privatan
`network::server::start_server()`	`pub mod server`	`fn`	`E0603`: funkcija je privatna

- Svaki element putanje mora biti javan: i moduli na putu, i sama funkcija.
- `pub mod` odlučuje da li se modul može imenovati, `pub fn` da li se funkcija može pozvati.
- Javna funkcija u privatnom modulu dostupna je spolja samo ako je izložena sa `pub use`.

Modul: pristup privatnom modulu

`primeri/intro-to-rust/project-example/src/bin/nocompile_private_module.rs`

```rust
// Intentional error: the module utils::internal is private (declared
 ↪  without pub),
// so a binary crate cannot access it. This binary is built only with
 ↪  `--features nocompile`.
use project_example::utils;

fn main() {
    utils::print_hello(); // OK: pub fn in a pub mod
    let text = utils::internal::decorate("text"); // error: module
     ↪  internal is private
    println!("{}", text);
}
```

`Izlaz`

```
error[E0603]: module `internal` is private
 --> project-example/src/bin/nocompile_private_module.rs:7:23
  |
7 |     let text = utils::internal::decorate("text"); // error: module
 ↪  internal is private
  |                       ^^^^^^^^  -------- function `decorate` is not
 ↪  publicly re-exported
  |                       |
  |                       private module
  |
```

```
note: the module `internal` is defined here
 --> project-example/src/utils/mod.rs:4:1
  |
4 | mod internal; // Private sub-module (no pub): visible only in utils
 ↪  and its descendants
  | ^^^^^^^^^^^^

For more information about this error, try `rustc --explain E0603`.
error: could not compile `project-example` (bin
 ↪  "nocompile_private_module") due to 1 previous error
```

Modul: vidljivost privatnih stavki

- Privatna stavka (bez `pub`) vidljiva je u modulu u kom je definisana i u svim njegovim potomcima.
  - Potomak vidi privatne stavke roditelja, dede i svih predaka (`super::`, `super::super::`).
  - Predak i sused ne vide privatne stavke potomka, odnosno suseda.
- Putanja do stavke mora da prođe kroz vidljive module: `pub fn` u privatnom modulu vidi samo onaj ko vidi taj modul.
- `pub` ne gleda ko poziva, već samo otvara stavku svima koji vide modul u kom se nalazi.

Vidljivost: ugnježđeni moduli

primeri/intro-to-rust/project-example/src/bin/visibility.rs

```rust
// Visibility of private items: an item is visible in the module that
 ↪  defines it
// and in all of its descendants, but not in its ancestors or siblings.
// Each commented-out line below is an access that the compiler rejects.
mod engine {
    fn engine_secret() -> &'static str { "engine_secret" }
    pub fn engine_public() -> &'static str { "engine_public" }

    mod kernel {
        fn kernel_secret() -> &'static str { "kernel_secret" }
        pub fn kernel_public() -> &'static str { "kernel_public" }

        mod inner {
            fn inner_secret() -> &'static str { "inner_secret" }
            pub fn inner_public() -> &'static str { "inner_public" }

            pub fn from_inner() -> String {
                format!("{} {} {}",
                    inner_secret(),                 // own item
                    super::kernel_secret(),         // parent
                    super::super::engine_secret())  // grandparent
            }
        }

        pub fn from_kernel() -> String {
            // let _ = inner::inner_secret();       // (1) E0603: function
             ↪  `inner_secret` is private
            format!("{} | {}", inner::inner_public(), inner::from_inner())
        }
    }
```

```
    mod helpers {
        pub fn from_helpers() -> String {
            // let _ = super::kernel::kernel_secret();          // (2)
             ↪  E0603: function `kernel_secret` is private
            // let _ = super::kernel::inner::inner_public();    // (3)
             ↪  E0603: module `inner` is private
            format!("{} {}", super::kernel::kernel_public(),
             ↪  super::engine_secret())
        }
    }

    pub fn from_engine() -> String {
        // let _ = kernel::kernel_secret();             // (4) E0603:
         ↪  function `kernel_secret` is private
        // let _ = kernel::inner::inner_public();       // (5) E0603:
         ↪  module `inner` is private
        // let _ = crate::api::cache::cache_public();   // (6) E0603:
         ↪  module `cache` is private
        format!("{} | {} | {}", kernel::from_kernel(),
         ↪  helpers::from_helpers(), engine_secret())
    }
}
```

Vidljivost: javni modul i `main`

primeri/intro-to-rust/project-example/src/bin/visibility.rs

```
pub mod api {
    mod cache {
        pub fn cache_public() -> &'static str { "cache_public" }
    }

    pub fn call() -> String {
        // let _ = crate::engine::engine_secret();       // (7) E0603:
         ↪  function `engine_secret` is private
        // let _ = crate::engine::kernel::kernel_public(); // (8) E0603:
         ↪  module `kernel` is private
        format!("{} {}", crate::engine::engine_public(),
         ↪  cache::cache_public())
    }
}

fn main() {
    println!("{}", engine::from_engine());
    println!("{}", api::call());
    // let _ = engine::engine_secret();           // (9) E0603: function
     ↪  `engine_secret` is private
    // let _ = engine::kernel::kernel_public();    // (10) E0603: module
     ↪  `kernel` is private
}
```

Izlaz

```
inner_public | inner_secret kernel_secret engine_secret | kernel_public
 ↪  engine_secret | engine_secret
engine_public cache_public
```

Vidljivost: zašto ne prolaze zakomentarisane linije

Linije	Pristup	Razlog
(1)	`kernel -> inner::inner_secret()`	roditelj ne vidi privatno u detetu
(2)	`helpers -> kernel::kernel_secret()`	sused ne vidi privatno u susedu
(3)	`helpers -> kernel::inner`	`inner` je privatan u `kernel`
(4), (5)	`engine -> kernel::kernel_secret()`, `kernel::inner`	predak ne vidi privatno u potomku
(6)	`engine -> api::cache`	`cache` je privatan u `api`
(7), (8)	`api -> engine::engine_secret()`, `engine::kernel`	`api` nije potomak od `engine`
(9), (10)	`main -> engine::engine_secret()`, `engine::kernel`	koren ne vidi privatno u `engine`

Upotreba use ključne reči: isti crate

- `use crate`
  - Ukoliko se modul nalazi u okviru istog crate-a.
- Na primer, `network::server` koristi `utils::helper` iz iste biblioteke:

`primeri/intro-to-rust/project-example/src/network.rs`

```rust
pub mod server {
    use crate::utils::helper; // Absolute path from the crate root:
     ↪  "utils" is in the same crate (library), so we start with crate::

    // A function in the "server" module
    pub fn start_server() {
        println!("Server started from the network::server module!");
        // Call a function from another module of the same crate
        helper::print_help();
    }
}
```

Upotreba use ključne reči: drugi crate

- `use project_name`, ukoliko se referencira:
  - Bibliotečki crate iz binarnog, u okviru istog paketa.
  - Bibliotečki crate iz eksternog integracionog testa (npr. datoteke u okviru tests direktorijuma).
  - Drugi crate (koji predstavlja zavisnost).
- Na primer, binarni crate `main` koristi biblioteku `project_example`:

`primeri/intro-to-rust/project-example/src/main.rs`

```rust
// Use the library crate defined in lib.rs
use project_example::config;
use project_example::network;
use project_example::utils;
```

Profili i objavljivanje

Profili izdanja

- Unapred definisani profili.
- Omogućavaju veću kontrolu nad opcijama kompajliranja:
- Glavni profili:

  - dev,
  - release.

primeri/intro-to-rust/Cargo.toml

```
# Profiles are set only in the workspace root; in a member's Cargo.toml
 ↪  they are ignored (cargo warns).
# These are the default values, written out to show the syntax.
[profile.dev]
opt-level = 0

[profile.release]
opt-level = 3
```

Cargo features

- Opcioni delovi crate-a koji se uključuju po potrebi.
- Definišu se u sekciji `[features]` u `Cargo.toml`.
  - U zagradama je šta feature još uključuje (druge feature-e, zavisnosti); `[]` ne uključuje ništa.
- Uključuju se opcijom `--features <ime>`.
- `required-features` izostavlja cilj (npr. binarni) dok feature nije uključen.

primeri/intro-to-rust/crate-sample/Cargo.toml

```
[features]
nocompile = []

[[bin]]
name = "nocompile_features"
required-features = ["nocompile"]
```

Cargo features: primer

- `cargo build` ga preskače, `cargo build --features nocompile` ga gradi.
- Primer namerno ne kompajlira, pa se prikazuje greška kompajlera.

primeri/intro-to-rust/crate-sample/src/bin/nocompile_features.rs

```
// Intentional error: this binary is built only with `--features
 ↪  nocompile`.
fn main() {
    let number: i32 = "text";
    println!("{number}");
}
```

Izlaz

```
error[E0308]: mismatched types
 --> crate-sample/src/bin/nocompile_features.rs:3:23
  |
3 |     let number: i32 = "text";
  |                 ---   ^^^^^^ expected `i32`, found `&str`
  |                 |
  |                 expected due to this

For more information about this error, try `rustc --explain E0308`.
error: could not compile `crate-sample-pdaj` (bin "nocompile_features")
 ↪  due to 1 previous error
```

Objavljivanje projekta na crates.io

- Crate-ovi koji se koriste u projektima se dobavljaju sa crates.io.
- Kako bi se objavio sopstveni crate neophodno je:
  - Napraviti (korisnu) dokumentaciju.
  - Kreirati API upotrebom `pub use`.
  - Podesiti nalog na crates.io.
  - Dodati metapodatke u crate.
  - Postaviti crate.

Primer crate-a: crate-sample

- Bibliotečki crate sa dve funkcije i binarnim crate-om koji ga koristi.

```
crate-sample/
├── Cargo.toml
└── src/
    ├── lib.rs
    ├── main.rs
    └── bin/
        └── nocompile_features.rs
```

Kreiranje (korisne) dokumentacije

- Dokumentacija je namenjena da programerima objasni kako da koriste crate, a ne kako je on implementiran:
  - Dokumentacioni komentar - `///`
  - Podrška za markdown.

primeri/intro-to-rust/crate-sample/src/lib.rs

```rust
mod example {
    // If pub mod was used here, instead of pub use below, the structure
     ↪  of the module would be known,
    // i.e. it would be possible to call functions from the module with
     ↪  crate_sample::example::add_one
    /// Adds one to the number given.
    ///
    /// # Examples
    ///
    /// ```
    /// let arg = 5;
    /// // We must call crate_sample::add_one instead of
     ↪  crate_sample::example::add_one because the
    /// // example module is private, and only its function was
     ↪  re-exported publicly via pub use.
    /// let answer = crate_sample::add_one(arg);
    ///
    /// assert_eq!(6, answer);
    /// ```
    pub fn add_one(x: i32) -> i32 {
        // If pub is omitted, the function is visible only inside this
         ↪  module
        x + 1
    }
}
```

Testovi u okviru dokumentacije

- U komentare je moguće dodati i primere koda (blok koda u `///` komentaru):
  - Pokretanje testova iz dokumentacije:
    - Pokreće izvršavanje testova.

```
$ cargo test --doc
```

  - Generisanje i otvaranje dokumentacije:

```
$ cargo doc --open
```

Korišćenje `pub use`

- Uz pomoć `pub use` konstrukta moguće je napraviti dostupnim:
  - Konstrukte unutar biblioteke:
    - Bez otkrivanja kako je biblioteka organizovana.

primeri/intro-to-rust/crate-sample/src/lib.rs

```
// This way, the functions from the modules can be used outside of them,
 ↪  by calling them with
// crate_sample::add_one, i.e. without knowing the structure of the
 ↪  modules
pub use example::add_one;
pub use example2::add_any;
```

`pub use`: korišćenje crate-a

- Funkcije se pozivaju bez poznavanja strukture modula: `add_one`, a ne `example::add_one`.

primeri/intro-to-rust/crate-sample/src/main.rs

```
use crate_sample::add_one;
fn main() {
    let num: i32 = 10;
    let result: i32 = add_one(num);
    println!("The result is: {}", result);
    let result2: i32 = crate_sample::add_any(num, 20);
    println!("The second result is: {}", result2);
}
```

Izlaz

```
The result is: 11
The second result is: 30
```

Podešavanje naloga na crates.io

- Napraviti nalog na crates.io (login via GitHub).
- Potvrditi mail adresu.
- Napraviti access token.

```
$ cargo login <YOUR-KEY>
```

Dodavanje metapodataka

- Neophodno je dodati ili izmeniti podatke u okviru Cargo.toml datoteke.
- Crate mora imati definisano:

  - Jedinstveno globalno ime (`name`).
  - Kratak opis (`description`).
  - Licenca (`license`).

primeri/intro-to-rust/crate-sample/Cargo.toml

```
[package]
name = "crate-sample-pdaj"
version = "0.1.0"
edition = "2021"
rust-version = "1.75"
description = "A sample crate"
license = "GPL-3.0"
```

Objavljivanje crate-a

```
$ cargo publish
```

- Objavljivanje je trajno!
  - Verzija se ne može prepisati.
  - Kod se ne može obrisati.
- Moguće je objaviti proizvoljan broj crate-ova.

Ažuriranje crate-a

- Izmeniti verziju nakon promene koda.
- Objaviti po standardnom metodu.

Yank crate-a

- Crate-ove nije moguće obrisati:
  - Ali je moguće onemogućiti dalje korišćenje:
    * Zabrana dodavanja u zavisnosti.
    * Dosadašnji paketi su u mogućnosti da nastave korišćenje.

```
$ cargo yank --vers <verzija>
```

Workspace

Workspace

- Moguće je “ugnježdavati” pakete
- Uz pomoć cargo workspace-a
- Na primer, workspace `intro-to-rust` sa dva paketa:

```
intro-to-rust/
├── Cargo.toml          // [workspace]
├── Cargo.lock
├── crate-sample/
│   ├── Cargo.toml
│   └── src/
└── project-example/
    ├── Cargo.toml
    └── src/
```

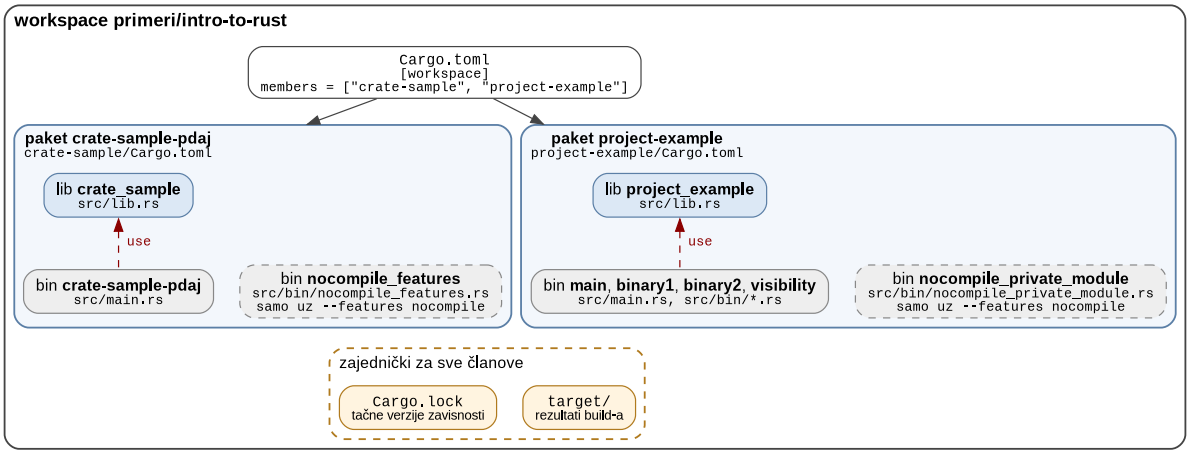
Workspace: članovi i izuzimanje

- Članovi (`members` u korenskom `Cargo.toml`) dele jedan `Cargo.lock` i jedan `target/` u korenu workspace-a.
- `members` prihvata i obrazac: `"*"` uzima sve podfoldere, pa svaki mora imati `Cargo.toml`.
- `exclude` izuzima foldere iz workspace-a (npr. `target`).
  - Paket čiji `Cargo.toml` već ima `[workspace]` je koren drugog workspace-a, pa ga roditelj workspace mora izuzeti (inače: `multiple workspace roots`).
- `cargo build -p <ime-paketa>` gradi jedan paket, a `--workspace` sve članove.
  - `-p` prima ime paketa iz `Cargo.toml`, a ne ime foldera: `-p crate-sample-pdaj` radi, `-p crate-sample` ne.

primeri/intro-to-rust/Cargo.toml

```toml
[workspace]
resolver = "2"
members = ["crate-sample", "project-example"]
```

Workspace: primer iz kursa



Slika 3: Workspace primeri/intro-to-rust

Cargo kao package manager

- Moguće je instalirati pakete putem cargo-a:

```
$ cargo install <naziv-paketa>
```

- Neophodno je u PATH dodati putanju
  - Do direktorijuma gde se nalaze bin datoteke
- Nakon toga pokretati paket kao i obično

Zadatak

Zadatak

- Pokrenuti terminal i proveriti instalaciju rustc-a i cargo-a.
- Napraviti novi projekat.
- Izmeniti podrazumevani binarni crate.
  - Predlog: Sumirati prvih 10^7 prirodnih brojeva korišćenjem iteriranja.
- Kompajlirati i pokrenuti program.
  - U debug režimu.

    - U release režimu.
- Izmeriti izvršavanja i uporediti vremena.

```
$ time ./executable
```

Predlog sadržaja main funkcije

```
fn main() {
    let mut sum: u64 = 0;
    for i in 1..=10_000_000 {
        sum += i;
    }
    println!("Sum: {}", sum);
}
```

Zadatak: napomene

- Izvršne datoteke su u `target/debug/` i `target/release/`.
- Debug profil koristi `opt-level = 0`, a release `opt-level = 3` (vidi Profili izdanja).
- Release verzija je znatno brža: optimizator pojednostavljuje petlju (novije verzije kompajlera mogu i da izračunaju zbir unapred).
- Ne povećavati broj iteracija bez razmišljanja: zbir mora da stane u `u64`.
- Pre merenja proveriti da je aktivna verzija 1.75.0: `rustc --version`.

Izvori

Izvori

- Rust Community. “The Rust Programming Language - the Rust Programming Language.” Rust-Lang.org, 2018, doc.rust-lang.org/book/.
- Rust Team. “Rust Programming Language.” Rust-Lang.org, 2018, www.rust-lang.org/.